JavaScript being JavaScript—is a constraint you have to enforce yourself. Each call to the reducer function returns a new instance of the state object that is separate from all the other instances.

This is somewhat more verbose than, you know, not doing this kind of change with a reducer, which obviously raises the question, Why use this structure at all? To be perfectly honest, this is a case where my own taste in software structures clashes a little bit. I find this pattern, as typically implemented in JavaScript, to be verbose. That said, the basic idea of making the central state immutable and only accessible via specific methods is a good one, and we'll stick with talking about the pattern, as you are likely to see it in other JavaScript code.

One problem with a reducer on its own is the possibility that different callers to the reducer might get out of sync. A solution to that is to also maintain a store. For our purposes here, a *store* is a centralized object that manages access to both a single source of central data and a related reducer. For our coin example, a store might look like this:

```
export class CoinStore {
  static state = {count: 0, value: 0}
  static getState(): { return state }
  static dispatch(action) {
    CoinStore.state = reducer(CoinStore.state, action)
    return CoinStore.state
  }
}
```

Again, we're holding off on TypeScript annotations. This code sets up a store with a single point of access:

```
CoinStore.dispatch({type: "AddPenny"})
CoinStore.dispatch({type: "AddNickel"})
const finalState = CoinStore.getState()
```

In this case, you know that your return value from `dispatch` is the current state, but typically you only ask for the state explicitly, otherwise you act on it only through actions.